

TIPE : Du jeu de taquin aux tableaux de Young

Camelia REKIOUA

28176

2023/2024

Introduction

Position du problème

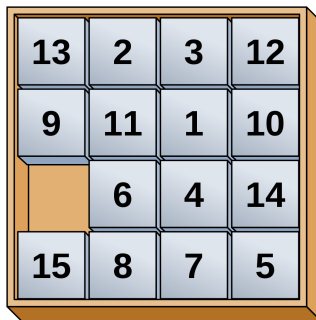


Figure 1 – Jeu de taquin mélangé

<https://fr.wikipedia.org/wiki/Taquin#/media/Fichier:15-puzzle-shuffled.svg>

Introduction

Position du problème

- Les configurations pouvant être résolues dans le jeu de taquin.

Introduction

Position du problème

- Les configurations pouvant être résolues dans le jeu de taquin.
- Résolution algorithmique



Figure 2 – Jeu de taquin résolu

<https://fr.wikipedia.org/wiki/Taquin#/media/Fichier:15-puzzle.svg>

Introduction

Position du problème

- Application du jeu de taquin sur les tableaux de Young.

Introduction

Position du problème

- Application du jeu de taquin sur les tableaux de Young.
- Munir l'ensemble des tableaux de Young semi-standards d'une structure de monoïde.

Plan de l'exposé

- 1 Permutations réalisables au jeu de taquin
- 2 Algorithme IDA*
- 3 Jeu de taquin de Schützenberger

Plan de l'exposé

- 1 Permutations réalisables au jeu de taquin
- 2 Algorithme IDA*
- 3 Jeu de taquin de Schützenberger

Problème de Sam Loyd

Un casse-tête mathématique

- Peut-on résoudre le jeu de taquin en partant d'une configuration quelconque ?



Figure 3 – Exemple de configuration du jeu de taquin.

<https://fr.wikipedia.org/wiki/Taquin#/media/Fichier:15-puzzle-loyd.svg>

Problème de Sam Loyd

Résolution

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	



4	1	3	2
5	6	7	8
9	10	11	12
13	14	15	

est représentée par la permutation : $(1, 4, 2)$

Problème de Sam Loyd

Résolution

Théorème 2.1

*Les permutations réalisables sur le jeu de taquin sont **exactement** les permutations paires.*

Plan de l'exposé

- 1 Permutations réalisables au jeu de taquin
- 2 **Algorithme IDA***
- 3 Jeu de taquin de Schützenberger

Résolution algorithmique

Quelques notations générales

On note :

- E l'ensemble des états/configurations possibles du jeu.

Résolution algorithmique

Quelques notations générales

On note :

- E l'ensemble des états/configurations possibles du jeu.
- s_0 la configuration initiale.

Résolution algorithmique

Quelques notations générales

On note :

- E l'ensemble des états/configurations possibles du jeu.
- s_0 la configuration initiale.
- $F \subset E$ l'ensemble des configurations gagnantes du jeu.

Résolution algorithmique

Quelques notations générales

On note :

- E l'ensemble des états/configurations possibles du jeu.
- s_0 la configuration initiale.
- $F \subset E$ l'ensemble des configurations gagnantes du jeu.
- $f : E \rightarrow \mathcal{P}(E)$ tel que pour tout s appartenant à E , $f(s)$ est l'ensemble des configurations atteignables à partir de s en une étape.

Résolution algorithmique

On modélise E par un graphe connexe.

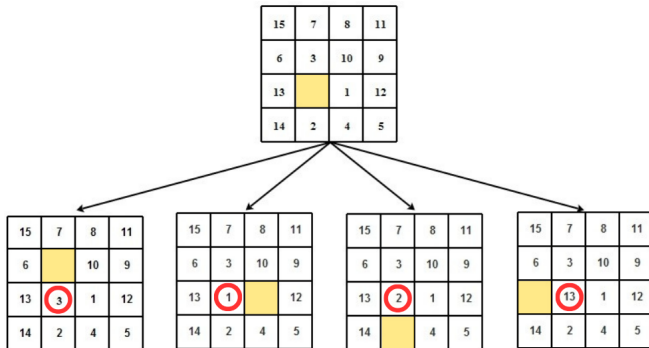


Figure 4 – Graphe modélisant le problème

Résolution algorithmique

Définitions

Définition 3.1

Le facteur de branchement b de E est défini pour tout $s \in E$ comme étant le cardinal de $f(s)$.

Nous supposons que b est constant.

Résolution algorithmique

Définitions

Définition 3.2

Un sommet s_p est à la profondeur p si il existe une séquence finie $s_0...s_p$ tel que pour tout $0 \leq i < p$, on a : $s_{i+1} \in f(s_i)$.
Si $s_p \in F$: $s_0...s_p$ est une solution du jeu.

On s'intéresse à la recherche de solutions optimales, c'est-à-dire de profondeur minimale.

Une première approche

Algorithme BFS

Fonction BFS(s_0)

déjà_vu $\leftarrow \emptyset$

à_traiter $\leftarrow \{s_0\}$

p $\leftarrow 0$

Tant que à_traiter n'est pas vide :

On retire s de à_traiter :

Si $s \in F$:

Renvoyer Vrai

Ajouter s à déjà_vu

Pour chaque état voisin t de s :

Si t n'est pas dans déjà_vu :

Ajouter t à à_traiter

p $\leftarrow p + 1$

Renvoyer Faux

Une première approche

Limites du BFS

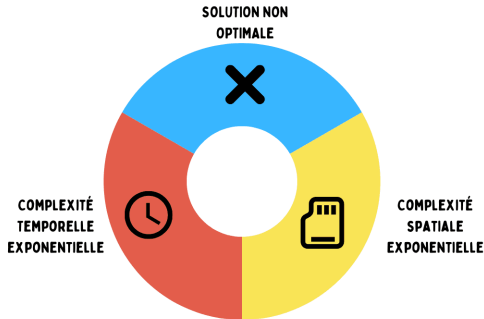


Figure 5 – Limites du BFS

Une première approche

Algorithme DFS

Fonction DFS(max,s,p)

Fonction DFS(max, s, p) :

Si $p > \text{max}$:

Renvoyer Faux

Si $s \in F$:

Renvoyer Vrai

Pour chaque voisin t de s :

Si DFS(max, t, p + 1) = Vrai :

Renvoyer Vrai

Renvoyer Faux

Utilisation : DFS(profondeur_max, s_0 , profondeur_initiale)

Une première approche

Limites du DFS

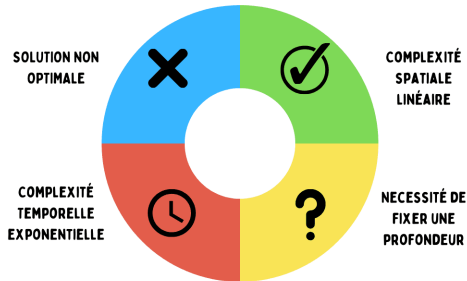


Figure 6 – Limites du DFS

Une approche efficace

IDDFS

Fonction IDDFS

Fonction IDDFS() :

$m = 0$

Tant que non DFS(max, e_0 , 0) :

$m = m + 1$

Renvoyer m

Une approche efficace

IDDFS

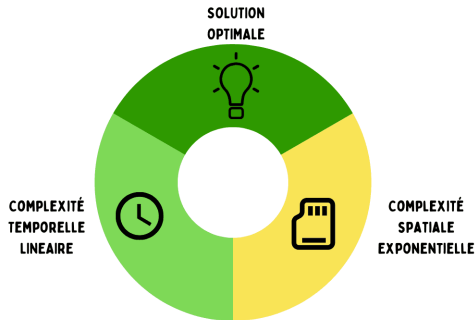


Figure 7 – Avantages du IDDFS

Utilisation d'une heuristique

IDA*

- Variante de l'algorithme IDDFS
- Utilisation d'une fonction heuristique

Utilisation d'une heuristique

IDA*

Définition 3.3

Une heuristique h est une fonction utilisée dans les algorithmes de recherche pour estimer le coût du chemin restant vers l'état but à partir d'un état donné dans un problème de recherche. Elle est dite admissible si elle ne surestime pas le coût du chemin restant vers l'état but à partir de l'état actuel.

Utilisation d'une heuristique

IDA*

- Coût total d'un chemin : $c(n) = g(n) + h(n)$
 $g(n)$: coût pour atteindre le noeud depuis la racine
 $h(n)$: estimation du coût pour atteindre l'état final.

Utilisation d'une heuristique

IDA*

- Coût total d'un chemin : $c(n) = g(n) + h(n)$
 $g(n)$: coût pour atteindre le noeud depuis la racine
 $h(n)$: estimation du coût pour atteindre l'état final.
- Interruption du parcours si le coût total du chemin dépasse un seuil donné.

Utilisation d'une heuristique

IDA*

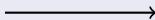
- Coût total d'un chemin : $c(n) = g(n) + h(n)$
 $g(n)$: coût pour atteindre le noeud depuis la racine
 $h(n)$: estimation du coût pour atteindre l'état final.
- Interruption du parcours si le coût total du chemin dépasse un seuil donné.
- Étant donné le seuil actuel, le seuil pour la prochaine itération est défini comme le minimum des coûts qui ont dépassé le seuil actuel.

Utilisation d'une heuristique

La distance de Manhattan

Exemple 3.1

4	1	3	2
5	11	14	6
8	10	7	12
13	9	15	



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

Dans ce cas la distance de Manhattan vaut :

$$1 + 2 + 0 + 3 + 0 + 2 + 1 + 4 + 2 + 0 + 2 + 0 + 0 + 3 + 0 = 20$$

Simulation

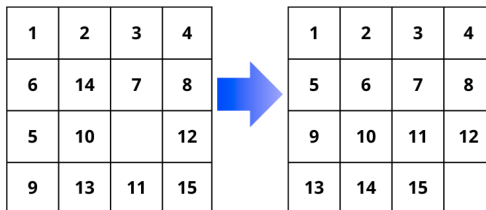


Figure 8 – Exemple de résolution

Simulation

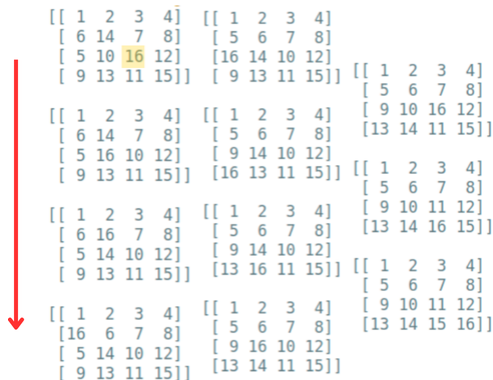


Figure 9 – Résolution avec IDA*

Plan de l'exposé

- 1 Permutations réalisables au jeu de taquin
- 2 Algorithme IDA*
- 3 Jeu de taquin de Schützenberger

Jeu de taquin de Schützenberger

Définitions

Définition 4.1

Soit $n \in \mathbb{N}$.

On appelle **partition** toute suite décroissante $(p_i)_{1 \leq i \leq n}$ d'entiers naturels telle que :

$$|\underline{p}| := \sum_{i=1}^{\infty} p_i < \infty$$

C'est-à-dire que la suite $\underline{p}$ est nulle à partir d'un certain rang.
De plus, si $|\underline{p}| = n$, alors $\underline{p}$ est une partition de n .

Jeu de taquin de Schützenberger

Définitions

Définition 4.2

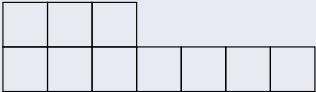
Un **diagramme de Young** de type $\underline{p}$ est un arrangement de cases, étroitement juxtaposées tel que les cases sont alignées sur la gauche et pour tout $i \in \llbracket 1, n \rrbracket$, on a p_i cases sur la ligne i . On le note $D(\underline{p})$.

Jeu de taquin de Schützenberger

Définitions

Exemple 4.1

On pose $\underline{p_1} = (7, 3)$ et $\underline{p_2} = (1^3)$.

Alors, $D(\underline{p_1}) =$  et $D(\underline{p_2}) =$ 

Jeu de taquin de Schützenberger

Définitions

Définition 4.3

Si k est un entier naturel non-nul, on note $[k] = \{1, 2, \dots, k\}$.

Un **tableau de Young** de type $\underline{p}$ est un **diagramme de Young** de type $\underline{p}$ dont les cases sont remplies avec des éléments de $[k]$.

De plus, si sur chaque ligne (lues de gauche à droite) les valeurs sont croissantes et sur chaque colonne (lues de bas en haut) les valeurs sont strictements croissantes : on dit que le tableau de Young est **semi-standard**.

Jeu de taquin de Schützenberger

Définitions

Exemple 4.2

On pose $\underline{p} = (3, 2, 1, 0)$

Alors, $T :=$

4		
2	3	
1	1	2

 est un tableau de Young semi-standard.

Mais, $T' :=$

4		
1	3	
1	1	2

 n'est pas semi-standard.

Jeu de taquin de Schützenberger

Définitions

On définit une relation d'ordre partiel $\leq$ sur les partitions :

$\underline{p} \leq \underline{q}$ si $p_1 \leq q_1, p_2 \leq q_2, \dots, p_n \leq q_n$

Jeu de taquin de Schützenberger

Définitions

Définition 4.4

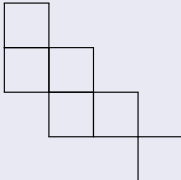
Si $\underline{q} \leq \underline{p}$, le diagramme gauche $D(\underline{p} \setminus \underline{q})$ de type $\underline{p} \setminus \underline{q}$ est obtenu à partir du diagramme $D(\underline{p})$ en supprimant les cases $D(\underline{q}) \subset D(\underline{p})$.

Jeu de taquin de Schützenberger

Définitions

Exemple 4.3

$$\underline{p} = (4, 3, 2, 1) \leq \underline{q} = (3, 1, 0, 0)$$

$$D(\underline{p} \setminus \underline{q}) =$$


Jeu de taquin de Schützenberger

Définitions

Définition 4.5

Un **tableau gauche** de type $\underline{p} \backslash \underline{q}$ est un **diagramme gauche** de type $\underline{p} \backslash \underline{q}$ dont les cases sont remplies avec des éléments de $[k]$.

Jeu de taquin de Schützenberger

Définitions

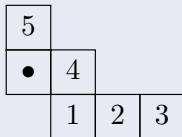
Définition 4.6

Un **tableau gauche pointé** de type $\underline{p} \backslash \underline{q}$ est un **diagramme gauche** de type $\underline{p} \backslash \underline{q}$ dont les cases **sauf une** sont remplies avec des éléments de $[k]$.

Jeu de taquin de Schützenberger

Définitions

Exemple 4.4



Jeu de taquin de Schützenberger

Définitions

Si on enlève la case vide :

5			
	4		
	1	2	3

Le tableau obtenu est toujours un tableau gauche, on dit alors que la case vide est **enlevable**.

Jeu de taquin de Schützenberger

Définitions

Exemple 4.5

5
•

•
4

4

1

2

3

est de type $(4, 2, 1) \setminus (1)$

5

4

4

1

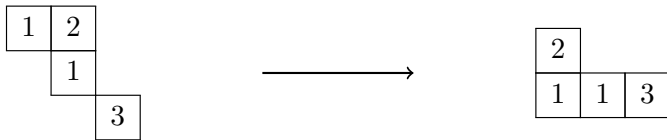
2

3

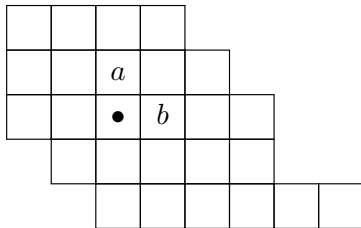
est de type $(4, 2, 1) \setminus (1, 1)$.

La case vide sera dite **intérieure**.

Principe du jeu de taquin de Schützenberger



Principe du jeu de taquin de Schützenberger



$$\begin{pmatrix} a & \\ \cdot & b \end{pmatrix}$$

- Remarque : s'il n'y a pas de valeurs correspondantes, on remplace la valeur de a ou b par ∞ .

Principe du jeu de taquin de Schützenberger

- on définit une opération qui transforme un tableau gauche pointé semi-standard avec une case vide intérieure en un tableau gauche pointé semi-standard avec une case vide extérieure :

Taquin
→

Principe du jeu de taquin de Schützenberger

- Principe de l'opération : $\xrightarrow{\text{Taquin}}$
 - Si $b = a = \infty$: la case vide est déjà une case vide extérieure.

Principe du jeu de taquin de Schützenberger

- Principe de l'opération : $\xrightarrow{\text{Taquin}}$
 - Si $b = a = \infty$: la case vide est déjà une case vide extérieure.
 - Sinon, tant que $b \neq e \neq \infty$:
 - si $a > b$: on échange

•

 et

b

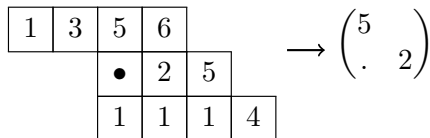
 - si $a \leq b$: on échange

•

 et

a

Principe du jeu de taquin de Schützenberger



Principe du jeu de taquin de Schützenberger

1	3	5	6		
		2	●	5	
		1	1	1	4

Principe du jeu de taquin de Schützenberger

1	3	5	6		
		2	5	●	
		1	1	1	4

Principe du jeu de taquin de Schützenberger

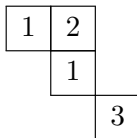
Soit T un tableau gauche semi-standard

On applique cet algorithme :

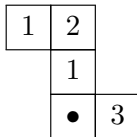
$$T \xrightarrow{\text{Ajout d'une case vide intérieure}} T' \xrightarrow{\text{Taquin}} T'' \xrightarrow{\text{Suppression de la case vide extérieure}} T_1$$

On nommera cette opération "un glissement" et on notera $T \Rightarrow T_1$.

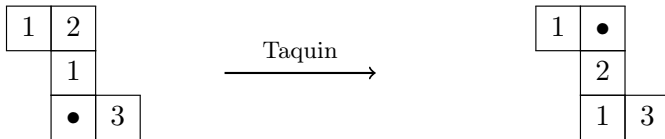
Principe du jeu de taquin de Schützenberger



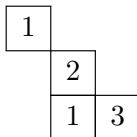
Principe du jeu de taquin de Schützenberger



Principe du jeu de taquin de Schützenberger



Principe du jeu de taquin de Schützenberger



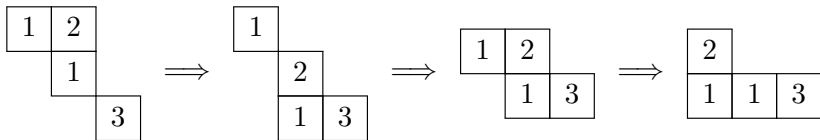
Principe du jeu de taquin de Schützenberger

- Le glissement de Schützenberger est réversible : avec la donnée du tableau final et de la case vide enlevée, il est possible de retrouver le tableau l'initial avec la case vide ajoutée.

Principe du jeu de taquin de Schützenberger

- On peut choisir une case vide intérieure du tableau obtenu après glissement et itérer le processus jusqu'à obtenir un tableau de Young semi-standard que l'on notera : $Rect(T)$.

Principe du jeu de taquin de Schützenberger



Principe du jeu de taquin de Schützenberger

Théorème 4.1

Etant donné un tableau gauche semi-standard, tous les choix possibles de la case vide intérieure mènent au même tableau rectifié.

Construction d'un produit sur les tableaux de Young semi-standards

Application du jeu de taquin

$$\begin{array}{|c|c|} \hline e & f \\ \hline a & b \\ \hline \end{array} \cdot \begin{array}{|c|c|c|} \hline y \\ \hline x \\ \hline u & v & w \\ \hline \end{array} = ?$$

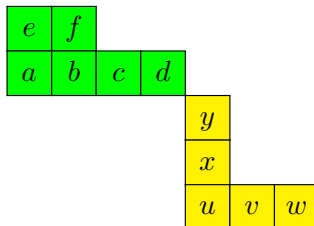
Construction d'un produit sur les tableaux de Young semi-standards

Application du jeu de taquin

<i>e</i>	<i>f</i>					
<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>			
				<i>y</i>		
				<i>x</i>		
				<i>u</i>	<i>v</i>	<i>w</i>

Construction d'un produit sur les tableaux de Young semi-standards

Application du jeu de taquin



Construction d'un produit sur les tableaux de Young

$$\begin{array}{|c|c|} \hline e & f \\ \hline a & b \\ \hline \end{array} \cdot \begin{array}{|c|} \hline y \\ \hline x \\ \hline u \\ \hline \end{array} \begin{array}{|c|c|c|} \hline v & w & \\ \hline \end{array} = \text{Rect}$$

$$\begin{array}{|c|c|} \hline e & f \\ \hline a & b \\ \hline \end{array} \begin{array}{|c|c|c|} \hline c & d & \\ \hline \end{array} \begin{array}{|c|} \hline y \\ \hline x \\ \hline u \\ \hline \end{array} \begin{array}{|c|c|c|} \hline v & w & \\ \hline \end{array}$$

Construction d'un produit sur les tableaux de Young semi-standards

Application du jeu de taquin

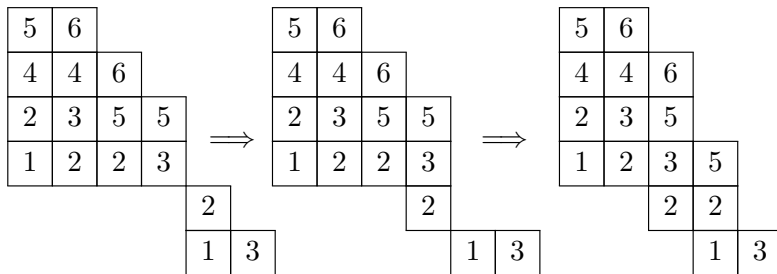
$$\begin{array}{|c|c|} \hline 5 & 6 \\ \hline 4 & 4 & 6 \\ \hline 2 & 3 & 5 & 5 \\ \hline 1 & 2 & 2 & 3 \\ \hline \end{array} \cdot \begin{array}{|c|c|} \hline 2 \\ \hline 1 & 3 \\ \hline \end{array} = ?$$

Construction d'un produit sur les tableaux de Young semi-standards

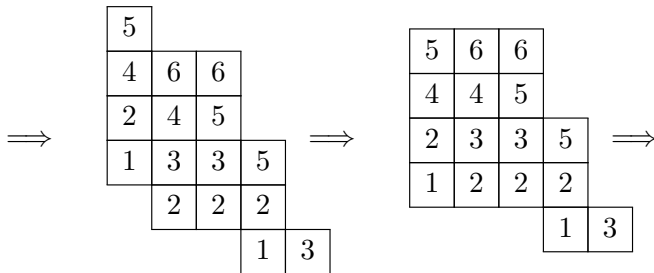
Application du jeu de taquin

5	6					
4	4	6				
2	3	5	●			
1	2	3	5			
		2	2			
				1	3	

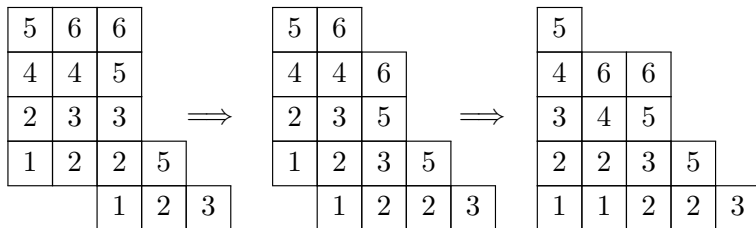
Principe du jeu de taquin de Schützenberger



Principe du jeu de taquin de Schützenberger



Principe du jeu de taquin de Schützenberger



Construction d'un produit sur les tableaux de Young semi-standards

Application du jeu de taquin

$$\begin{array}{|c|c|} \hline 5 & 6 \\ \hline 4 & 4 & 6 \\ \hline 2 & 3 & 5 & 5 \\ \hline 1 & 2 & 2 & 3 \\ \hline \end{array} \cdot \begin{array}{|c|} \hline 2 \\ \hline 1 & 3 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 5 \\ \hline 4 & 6 & 6 \\ \hline 3 & 4 & 5 \\ \hline 2 & 2 & 3 & 5 \\ \hline 1 & 1 & 2 & 2 & 3 \\ \hline \end{array}$$

Construction d'un produit sur les tableaux de Young semi-standards

Application du jeu de taquin

Théorème 4.2

L'ensemble des tableaux de Young semi-standard muni du produit $\cdot$ est un monoïde d'élément neutre le tableau vide $\emptyset$.

Correspondance de RSK

La correspondance de RSK (Robinson-Schensted-Knuth) permet d'identifier un mot dans un alphabet A .

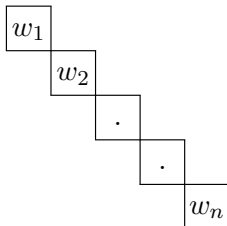
Correspondance de RSK

Soit A un alphabet et $w \in A$.

Grâce au jeu de taquin, il est possible de construire un tableau représentatif semi-standard sur A représentatif de w : le **tableau d'insertion**.

Correspondance de RSK

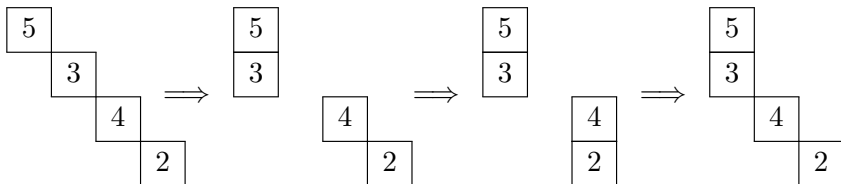
- Si $w = w_1 w_2 \dots w_n$ avec n la longueur de w , on construit le tableau de de forme anti-diagonale :



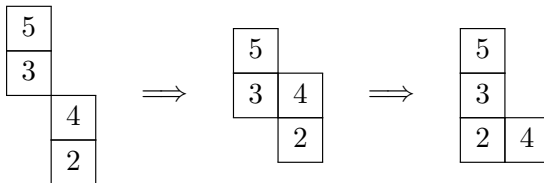
- On applique le jeu de taquin de Schützenberger à ce tableau pour obtenir le tableau d'insertion.

Correspondance de RSK

$$w = 5342$$



Correspondance de RSK



Correspondance de RSK

On pose

$$\begin{aligned}\sigma : A &\rightarrow \text{SST} \\ w &\mapsto T(w)\end{aligned}$$

où SST est l'ensemble des tableaux semi-standard sur A et $T(w)$ est le tableau d'insertion de w .

σ est surjective, mais pas injective. En effet, les mots 121 et 211 ont la même image par σ .

Conclusion

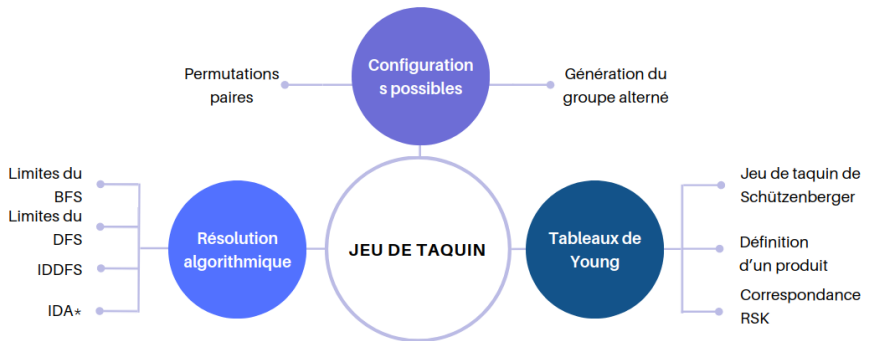


Figure 10 – Schéma récapitulatif

Problème de Sam Loyd

Résolution

Démonstration

Un déplacement horizontal ne change pas l'ordre des numéros, cela correspond à Id .

Un déplacement vertical correspond à un 3-cycle.

Donc, les permutations réalisables au jeu de taquin sont des produits de permutations paires.

Étude des configurations possibles

Une réponse plus complète

Lemme 6.1

Soit $\sigma \in \mathfrak{S}_n$, $n \in \mathbb{N}$ et $i \in \{1, 2, \dots, 13\}$

Si σ est réalisable sur le jeu de taquin, alors $\sigma \circ (i, i + 1, i + 2)$ l'est aussi.

Étude des configurations possibles

Une réponse plus complète

Démonstration

Il suffit de montrer que l'on peut réaliser $(i, i + 1, i + 2)$ avec $i \in \{1, 2, \dots, 13\}$

Les cycles $(3, 4, 5)$ et $(1, 2, 3, 4, 5, 6, 7)$ sont réalisables.

La composée de $(3, 4, 5)$ avec les puissances successives de $(1, 2, 3, 4, 5, 6, 7)$ montre que les cycles $(i, i + 1, i + 2)$ avec $i \in \{1, 2, \dots, 5\}$ sont réalisables.

De même en se plaçant à la deuxième ligne, on montre que les cycles $(i, i + 1, i + 2)$ avec $i \in \{5, 6, \dots, 9\}$ sont réalisables.

Puis que les cycles $(i, i + 1, i + 2)$ avec $i \in \{9, 10, \dots, 13\}$ le sont aussi.

Étude des configurations possibles

Une réponse plus complète

Lemme 6.2

Soit $n \geq 2$ et $i \in \{1, 2, \dots, n-2\}$

Les trois cycles $(i, i+1, i+2)$ engendrent le groupe alterné A_n .

- Ce résultat conclut la démonstration du théorème 2.1

Une première approche

Limites du BFS

- Le BFS ne garantit pas la découverte de solution de profondeur minimale.
- Dans le pire des cas, le parcours en profondeur pourrait nécessiter la génération de tous les nœuds jusqu'à la profondeur p :
$$b + b^2 + b^3 + \dots + b^p = O(b^p)$$
- Le BFS stocke tous les nœuds d'une profondeur avant de passer à la suivante, le nombre minimal de nœuds à stocker pour atteindre une profondeur p est b^{d-1} , d'où une complexité spatiale de $O(b^d)$.

Une première approche

Limites du DFS

- Le DFS ne garantit pas la découverte de solution de profondeur minimale.
- Seul le chemin du nœud initial au nœud actuel doit être stocké, d'où une complexité spatiale en $O(p)$.
- Complexité temporelle : $O(b^p)$
- Il faut imposer une profondeur de coupure arbitraire pour que l'algorithme termine. Si l'estimation est trop basse, l'algorithme se termine sans trouver de solution et si l'estimation de la profondeur est trop élevée, il y a un prix élevé en temps d'exécution.

Une approche efficace

IDDFS

- Étant donné que IDDFS n'explore jamais plus profondément que la profondeur p , l'espace qu'il utilise est en $O(p)$

Une approche efficace

IDDFS

- si $b > 1$, l'algorithme IDDFS a une complexité spatiale en $O(b^p)$

Les noeuds à la profondeur $p - k$ avec $k \in \llbracket 0, p \rrbracket$ sont générés $k + 1$ fois.

Nombre total de noeuds générés :

$$\begin{aligned} \sum_{k=0}^p (k+1)b^{(d-k)} &= b^p \sum_{k=0}^p (k+1)\left(\frac{1}{b}\right)^k \\ &< b^p \sum_{k=0}^{\infty} (k+1)\left(\frac{1}{b}\right)^k = b^p \left(1 - \frac{1}{b}\right)^{-2} = O(b^p) \end{aligned}$$

Une approche efficace

IDDFS

Théorème 6.1

L'algorithme IDDFS renvoie une solution optimale.

Une approche efficace

IDDFS

Lemme 6.3

La fonction $DFS(max, s_0, 0)$ renvoie VRAI si et seulement si une solution de profondeur $p \leq m$ existe.

Une approche efficace

IDDFS

Démonstration du lemme

Supposons qu'il existe une solution de profondeur $p \leq m$: $s_0 \dots s_p$. Alors $DFS(max, s_p, p)$ va renvoyer VRAI. Donc $DFS(max, s_{p-1}, p-1)$ également, et par itération, $DFS(max, s_0, 0)$ également.

Réciproquement, si $DFS(max, s_0, 0)$ renvoie VRAI, il y a deux cas :

Soit s_0 est une configuration finale donc la solution existe et a une longueur nulle.

Soit, pour un successeur s_1 de s_0 , $DFS(max, s_1, 1)$ renvoie vraie. On peut alors appliquer le même raisonnement de manière itérative pour construire la solution qui sera (par construction) de profondeur p finie bornée par m .

Une approche efficace

IDDFS

Démonstration

Supposons qu'il existe une solution, on fixe m la profondeur d'une solution optimale.

Pour tout $k < m$ l'algorithme ne trouve pas de solution.

Par application du lemme précédent, DFS($m, s_0, 0$) renvoie vrai et la profondeur renvoyée est bien celle d'une solution optimale.

Une approche efficace

IDDFS

Le désavantage de l'IDDFS est qu'il effectue des calculs inutiles avant d'atteindre la profondeur souhaitée. Cela peut sembler inefficace au premier abord. Cependant, cela n'affecte pas la croissance asymptotique du temps d'exécution pour des arbres exponentiels comme celui du jeu de taquin.

Une approche efficace

IDDFS

Théorème 6.2

L'algorithme IDDFS est asymptotiquement optimal en termes de temps et d'espace.

Une approche efficace

IDDFS

Démonstration

- **Complexité spatiale :**

Le nombre de nœuds à la profondeur p est b^p . Supposons qu'il existe un algorithme qui examine moins de b^p nœuds. Alors, il doit exister au moins un nœud à la profondeur p qui n'est pas examiné par cet algorithme. La seule solution pourrait être placée à ce nœud, et donc l'algorithme proposé échouerait.

Une approche efficace

IDDFS

Démonstration

- **Complexité temporelle :**

Notons que tout algorithme qui utilise $f(n)$ temps doit utiliser au moins $k \cdot \log f(n)$ espace pour une certaine constante k . La raison en est que l'algorithme doit parcourir $f(n)$ états distincts avant de boucler ou de se terminer, et donc doit être capable de stocker autant d'états distincts. Puisque stocker $f(n)$ états nécessite $\log f(n)$ bits, et comme $\log b^p = p \cdot \log b$, tout algorithme doit utiliser $k \cdot d$ d'espace, pour une certaine constante k .

Programme python

```
1 import numpy as np
2 # Mouvements possibles (haut, bas, gauche, droite)
3 MOVES = {
4     'U': (-1, 0),
5     'D': (1, 0),
6     'L': (0, -1),
7     'R': (0, 1)
8 }
9
10 class PuzzleNode:
11     # Initialise un noeud du puzzle.
12     def __init__(self, state, parent=None, move=None, cost=0, heuristic=0):
13         self.state = state
14         self.parent = parent
15         self.move = move
16         self.cost = cost
17         self.heuristic = heuristic
18
19     def __lt__(self, other):
20         return self.cost + self.heuristic < other.cost + other.heuristic
21
```

Figure 11 – Code python 1

Programme python

```
22 def manhattan_distance(state, goal):
23     #Calcule la distance de Manhattan entre l'état actuel et l'état but.
24     distance = 0
25     for value in range(1, len(state) * len(state[0])):
26         x1, y1 = np.where(state == value)
27         x2, y2 = np.where(goal == value)
28         distance += abs(x1 - x2) + abs(y1 - y2)
29     return distance
```

Figure 12 – Code python 2

Programme python

```
31 def generate_neighbors(node, goal):
32     #Génère les voisins d'un noeud en appliquant les mouvements possibles.
33     neighbors = []
34     state = node.state
35     blank_pos = np.where(state == 16)
36     x, y = blank_pos[0][0], blank_pos[1][0]
37
38     for move, (dx, dy) in MOVES.items():
39         new_x, new_y = x + dx, y + dy
40         if 0 <= new_x < state.shape[0] and 0 <= new_y < state.shape[1]:
41             new_state = state.copy()
42             new_state[x, y], new_state[new_x, new_y] = new_state[new_x, new_y], new_state[x, y]
43             new_cost = node.cost + 1
44             new_heuristic = manhattan_distance(new_state, goal)
45             neighbors.append(PuzzleNode(new_state, node, move, new_cost, new_heuristic))
46     return neighbors
```

Figure 13 – Code python 3

Programme python

```
48 def count_inversions(state):
49     #Compter le nombre d'inversions dans la permutation des tuiles.
50     new = state.flatten()
51     inversions = 0
52     for i in range(len(new)):
53         for j in range(i + 1, len(new)):
54             if new[i] != 16 and new[j] != 16 and new[i] > new[j]:
55                 inversions += 1
56     return inversions
57 def is_solvable(state):
58     #Vérifier si la configuration est résoluble.
59     inversions = count_inversions(state)
60     blank_pos = np.where(state == 16)
61     blank_row_from_bottom = state.shape[0] - blank_pos[0][0]
62     if state.shape[0] % 2 == 1:
63         return inversions % 2 == 0
64     else:
65         if blank_row_from_bottom % 2 == 1:
66             return inversions % 2 == 0
67         else:
68             return inversions % 2 == 1
```

Figure 14 – Code python 4

Programme python

```

70 def ida_star(start_state, goal_state):
71     #Implémente l'algorithme IDA* pour résoudre le puzzle.
72     threshold = manhattan_distance(start_state, goal_state)
73     def search(path, g, threshold):
74         #Fonction récursive de recherche pour l'algorithme IDA*.
75         node = path[-1]
76         f = g + node.heuristic
77         if f > threshold:
78             return f
79         if np.array_equal(node.state, goal_state):
80             return path
81         min_threshold = float('inf')
82         for neighbor in generate_neighbors(node, goal_state):
83             if not path or not np.array_equal(neighbor.state, path[-1].state):
84                 path.append(neighbor)
85                 result = search(path, g + 1, threshold)
86                 if isinstance(result, list):
87                     return result
88                 if result < min_threshold:
89                     min_threshold = result
90                 path.pop()
91         return min_threshold
92
93     start_node = PuzzleNode(start_state, heuristic=manhattan_distance(start_state, goal_state))
94     path = [start_node]
95     while True:
96         result = search(path, 0, threshold)
97         if isinstance(result, list):
98             return result
99         if result == float('inf'):
100             return None
101         threshold = result

```

Figure 15 – Code python 5

Insertion de Schensted

L'insertion de Schensted est un algorithme permettant d'insérer une valeur λ au sein d'un tableau de Young T .

Principe :

- Si λ est supérieur ou égal à tous les éléments de la première ligne, l'ajouter à la fin de cette ligne.
- Sinon, remplacer l'élément le plus petit de la ligne qui est supérieur à λ , puis insérer l'élément retiré dans la ligne suivante.
- Répéter jusqu'à ce qu'il n'y ait plus d'éléments à déplacer.

On notera cette opération $\lambda \rightarrow T$.

Insertion de Schensted

Notons que l'insertion de Schensted est une opération réversible.

Insertion de Schensted

```
1 def insertion(l, p):
2     """
3     Insère l'élément p dans la liste triée l et éjecte l'élément le plus grand
4     qui est plus petit que p.
5     """
6     result = []
7     added = False
8     ejected = p
9
10    for elem in l:
11        if elem > p and not added:
12            # Ajoute p à la liste et éjecte l'élément courant
13            result.append(p)
14            ejected = elem
15            added = True
16        else:
17            result.append(elem)
18
19    if not added:
20        # Si p est le plus grand élément, il est ajouté à la fin de la liste
21        ejected = None
22        result.append(p)
23
24    return [result, ejected]
```

Figure 17 – Code python 6

Insertion de Schensted

```
26 def schensted(l, p):
27     """
28     Applique l'algorithme d'insertion de Schensted pour insérer l'élément p
29     dans le tableau de Young représenté par une liste de listes triées.
30
31     """
32     eject = p
33
34     for i in range(len(l)):
35         if eject is None:
36             # Si rien n'est éjecté, on a fini
37             return l
38         # Insère l'élément éjecté dans la prochaine ligne
39         l[i], eject = insertion(l[i], eject)
40     if eject is not None:
41         # Si un élément est toujours éjecté, ajoute une nouvelle ligne
42         l.append([eject])
43
44     return l
```

Figure 18 – Code python 7

Insertion de Schensted

```
60 def max_smaller(l, p):
61     #Trouve le plus grand élément dans l qui est plus petit que p.
62     result= []
63     for elem in l :
64         if elem<p :
65             result.append(elem)
66     if result == [] :
67         return l[-1]
68     return max(result)
69
70 def deletion(l, p):
71     #Supprime l'élément dans la liste triée l et remplace l'élément le plus
72     #grand qui est plus petit que p.
73     result = []
74     added = False
75     eject = max_smaller(l, p)
76     for elem in l[::-1]:
77         if elem == eject and not added:
78             result.append(p)
79             added = True
80         else:
81             result.append(elem)
82     return [result[::-1], eject]
83
84 def inversion_schensted(l, p, index):
85     #Applique l'algorithme d'inversion de Schensted pour supprimer l'élément p
86     #dans le tableau de Young représenté par une liste de listes triées.
87     eject = p
88     work = l
89     work[index].remove(p)
90     for i in range(len(l)):
91         if len(l)-1-i < index :
92             result = deletion(l[len(l)-1-i], eject)
93             l[len(l)-1-i], eject = result [0], result [1]
94     return [list(filter(None, work)), eject]
```

Figure 19 – Code python

Insertion de Schensted

Grâce à l'insertion de Schensted, nous pouvons construire le tableau de Young représentatif d'un mot en insérant les lettres du mot dans un tableau vide.

Correspondance RSK

Soit A un alphabet.

On peut définir donc la bijection :

$$\begin{aligned}\sigma_+ : A &\longrightarrow SST \times ST \\ m &\longmapsto (W(m), P(m))\end{aligned}$$

où

- SST est l'ensemble des tableaux semi-standard sur A
- ST l'ensemble des tableaux standard sur $[k]$ avec k la longueur de m .
- $W(m)$ le tableau semi-standard représentatif de m et $P(m)$ le tableau standard issu de l'ordre placement des lettres de m .